



Google Summer of Code



Cuneiform Digital
Library Initiative

Scalable Email Infrastructure

Google Summer of Code -2026

Mentors: Vishv Kakadiya (supported by: Nisheal John)

Name	Gutha Sonika Chowdary
Email	gsonikachowdary@gmail.com
Phone	+91 7989757750
GitLab	https://gitlab.com/sonikagutha
LinkedIn	https://www.linkedin.com/in/sonika-chowdary-gutha/
Location	Hyderabad, India
Year	2nd Year
Branch	Computer Science
College	BITS, Hyderabad
Contributions	!1148 !1151 (merged)

Table of Contents

- 1. Abstract**
- 2. Interest**
- 3. Existing Email Infrastructure**
- 4. Problem Statement**
- 5. Proposed Solution**
- 6. Implementation Plan**
 - Decouple Email Delivery
 - Email Job Table Migration
 - Email Queue Services
 - Background worker
 - Retry Logic
 - Email Template System
 - Admin Dashboard
 - Rate limiting
 - Health Check System
 - Testing Strategy
 - Documentation
- 7. Milestones**
- 8. Timeline**
- 9. Deliverables**
- 10. Contribution**
- 11. Experience**
- 12. Why me?**
- 13. Use Of LLM**
- 14. Other commitments during summer**

Abstract

The CDLI framework sends transactional emails synchronously inside request handlers. This means user-facing operations like account registration and password reset are blocked until SMTP negotiation, DNS resolution, and relay acceptance all complete. When any step in that chain is slow or unavailable, the user request hangs. When it fails, the email is silently lost with no retry and no audit trail.

This project replaces that synchronous path with an asynchronous pipeline built on infrastructure already present in the repository. Emails are enqueued as database-persisted jobs and delivered by a background worker. Users get immediate responses. Failures are visible, retryable, and logged. Administrators gain a dashboard with queue health metrics and manual retry controls.

Interest

My interest in open source is from a simple realization: the tools I use every day, from Python and Django to PostgreSQL, exist because people chose to build in public and share their work. That changed how I think about software. Instead of seeing code as something written only for classes or internships, I started seeing it as something that can become genuinely useful to others.

GSoC feels like the right opportunity for me to take that step seriously and contribute to a real project in a meaningful way.

I set up the CDLI framework locally and started reading through the codebase — and the more I understood how emails were being fired directly from request handlers with no safety net, the more I wanted to be the one to fix it. This is the kind of work where you can point at something real at the end and say it works better now, and that matters to me more than building something impressive from scratch

Existing Email Infrastructure

Before proposing any changes, it is important to document exactly what the framework already does and what tools are already in place. Every design decision in this proposal builds directly on what exists.

2.1 Where Emails Are Triggered Today ?

There are two places in the codebase where emails are sent synchronously during the request lifecycle:

ForgotController.php

File: **app/cake/src/Controller/ForgotController.php**

The password reset flow calls the mailer directly at the end of the controller action. The HTTP response cannot be returned until this call returns.

```
try {  
    $this->getMailer('User')->send('resetPassword', [$user, $url]);  
    $this->Flash->success(__('Check you email for password reset link!'));
```

If the SMTP connection is slow or times out, the user sees a slow or failed request. There is no retry if this call throws an exception.

UsersTable.php — afterSaveCommit callback

File: **app/cake/src/Model/Table/UsersTable.php**

Two emails are sent immediately after a user record is committed to the database. The afterSaveCommit lifecycle hook fires them both **synchronously**:

```
try {  
    $this->getMailer('User')->send('adminNewUser', [$entity]);  
    $this->getMailer('User')->send('welcome', [$entity]);
```

A third action, adminCrowdsourcingPrivilege, follows the same pattern when privilege levels change. All three block the save operation from completing its response cycle.

2.2 The Mailer and Template Layer

The mailer class already exists at **app/cake/src/Mailer/UserMailer.php**. It defines four actions:

- welcome — sent to new users on registration
- resetPassword — sent when a user requests a password reset
- adminNewUser — sent to the administrator when any new user registers
- adminCrowdsourcingPrivilege — sent when user privilege is elevated

HTML templates live under **app/cake/templates/email/html/** and plain-text templates under **app/cake/templates/email/text/**. The rendering layer is complete and correct. What is missing is the async delivery orchestration around it.

Existing templates such as `resetpassword.php` and `welcome.php` are standalone content blocks with no shared layout, header, or footer. Each template handles its own HTML structure independently, which causes inconsistent styling across email types.

2.3 Task and Queue Infrastructure

File: **app/cake/src/Controller/Component/TaskComponent.php**

A Redis-backed task component already exists. It connects to Redis on database index 9 and exposes an `enqueue()` method that checks for duplicate job keys before inserting. This is currently used for generic background tasks, not email delivery. The infrastructure is present — it does not yet serve the email pipeline.

Redis is already configured in the project's Docker Compose environment files (**dev/docker-compose.dev.yml**, **dev/docker-compose.mpi.yml**, and **dev/docker-compose.gea.yml**). No new queue infrastructure needs to be introduced.

Beyond this component, the project already has background workers running for other tasks such as image processing (used in `Admin/UploadsController`). Here we are using the existing worker patterns for the email pipeline rather than introducing a separate processing architecture.

2.4 Postfix Relay

File: **dev/docker-compose.yml**

A Postfix relay container is already defined using the `juanluisbaptiste/postfix:1.8.0` image. It is configured through environment variables:

- `CDLI_POSTFIX_SMTP_SERVER`
- `CDLI_POSTFIX_SMTP_PORT`
- `CDLI_POSTFIX_SMTP_USERNAME`
- `CDLI_POSTFIX_SMTP_PASSWORD`

The application already points its SMTP transport at this Postfix container via EMAIL_TRANSPORT_DEFAULT_URL in the environment. The relay boundary between the application and the upstream provider is operational.

What is Postfix ?

Postfix acts as the local mail transfer agent (MTA), relaying outbound SMTP traffic from the application to the upstream SMTP provider.

In simple analogy like a postman delivering the posts

2.5 Sender Configuration

File: **app/cake/config/app.php**

A central sender identity is already defined:

```
'from' => 'no-reply@cdli.network'  
'replyTo' => 'cdli-support@ames.ox.ac.uk'
```

The Postfix relay is documented to accept senders in the `*@cdli.network` domain. This policy is not yet enforced at the application level — individual mailer actions can override the sender arbitrarily.

Summary: The mailer layer already exists, Postfix is already configured as the relay boundary, Redis-backed task infrastructure is already present, and worker patterns already exist elsewhere in the project. What does not yet exist is a dedicated email background pipeline: asynchronous email dispatch, a persistent email job table, retry logic, operational visibility, shared email layouts, health checks, and consistent sender-policy enforcement.

Problem Statement

With the existing infrastructure documented, the specific problems it creates become precise rather than general.

Email delivery blocks user requests

Calling `->send()` within the request lifecycle introduces unnecessary latency and tightly couples user-facing operations with external SMTP dependencies.

Any delay in SMTP negotiation, DNS resolution, or Postfix queue processing directly impacts response time and can lead to timeouts, especially in critical flows such as registration and password reset.

Failures are silent and unrecoverable

If `->send()` throws an exception due to a transient SMTP error, the error is logged and the email is gone. There is no retry queue. A user who triggers a password reset at the moment the upstream SMTP provider returns a temporary 421 response receives no email and has no way to know why. The email does not reappear when the provider recovers.

No persistent record of email state

There is no table or model tracking whether an email was attempted, delivered, retried, or failed. The only record is in log files, which are not structured for querying. It is impossible to answer: how many emails failed today, which recipient, which template, what was the error?

No operational visibility for administrators

There is no admin interface for email operations. An administrator cannot see the queue backlog, inspect a failed job, manually retry a lost email, or understand whether the worker is running. Diagnosing a delivery problem requires reading raw log output.

Email templates are inconsistent and unmaintained

Each template is a standalone HTML file with no shared layout, branding, or structural consistency. Updating the header or footer requires editing every template individually. The templates are simple standalone files and do not yet use a shared email-safe layout system with reusable structure, branding, and maintainable client-friendly styling patterns.

Sender policy is not enforced

The central sender identity in `app.php` can be overridden by any mailer action. This risks sending emails from addresses outside the `cdli.network` domain, which breaks SPF/DMARC alignment and reduces deliverability.

Rate limiting is incomplete

The existing per-user cooldown in `ForgotController.php` covers only one flow and is hardcoded. There is no IP-level protection. An attacker can enumerate whether an email address exists in the system by observing different error messages for known and unknown accounts.

Health of the email pipeline is invisible

If the email worker stops running, if Redis becomes unreachable, or if Postfix stops accepting connections, there is no alert and no monitoring endpoint. The failure is discovered only when users report missing emails.

Docker: Add a new service, `email_worker`, to the existing CakePHP + Redis + MariaDB + Postfix stack, responsible for asynchronous email processing.

Jobs: Managed using Redis for queue dispatch and the `email_jobs` table in MariaDB for durable persistence and state tracking.

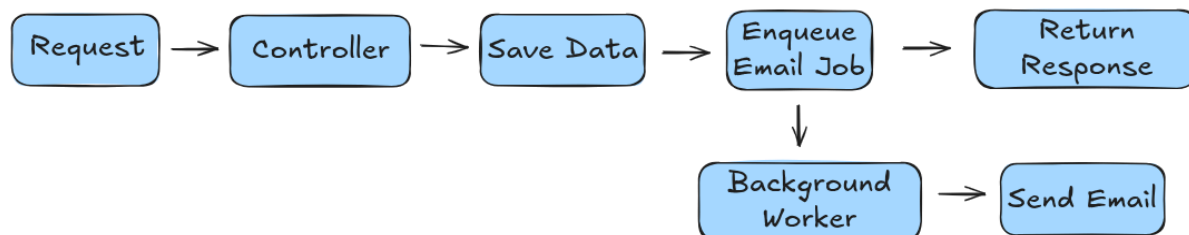
Scheduling: Handled by the worker via periodic scanning of retryable jobs based on the `next_retry_at` timestamp, ensuring controlled retries and fault tolerance.

Proposed Solution

Current Implementation Flow:



Proposed Implementation Flow:



4.1 Move email delivery to background processing

Emails will no longer be sent during the request lifecycle. Instead, the application will enqueue email jobs and return the response immediately, while a background worker delivers the message asynchronously.

4.2 Add persistent job tracking and retries

A new `email_jobs` table will store delivery state, retry count, timestamps, and failure details. This will allow retries with backoff and prevent transient delivery failures from silently dropping important emails.

4.3 Reuse the existing mailer and Postfix infrastructure

The existing CakePHP mailers and templates will continue to define the message content, while Postfix will remain the SMTP relay boundary. The project will build on the current Redis-based worker pattern rather than introducing a separate queueing stack.

4.4 Improve visibility and maintainability

The project will add an admin dashboard for monitoring pending, sent, retrying, and failed emails, along with shared email layouts, stronger rate limiting, sender-policy enforcement, and health checks for the mail pipeline.

4.5 Expected outcome

The final result will be a faster, more reliable, and more observable email system that improves user experience while remaining aligned with the framework's existing architecture.

Implementation Plan

This section explains how each component will be built, which files will change, and how the new pieces integrate with the existing codebase.

5.0 Decouple email delivery from the request cycle

Replace the **direct** `->send()` calls in **ForgotController.php** and **UsersTable.php** with a call to a new **EmailQueueService**.

The service writes a job record to the `email_jobs` database table and pushes the job ID to a Redis list. The controller returns its HTTP response immediately. The user never waits for SMTP.

5.1 Email Job Table Migration

A new **email_jobs table** becomes the source of truth for every outgoing email. Every job carries its full lifecycle — mailer name, action, recipient, payload, status, attempt count, error details, and timestamps. Failed emails are retained for inspection and manual retry rather than disappearing into logs.

Job status values follow a defined state machine:

- pending — written by EmailQueueService, awaiting worker pickup
- processing — claimed by a worker via `SELECT FOR UPDATE`
- sent — accepted by the configured mail transport / relay.
- retrying — temporary failure, scheduled for next attempt
- failed — exceeded max attempts, retained for admin review

File to create:

`config/Migrations/YYYYMMDDHHMMSS_CreateEmailJobsTable.php`

A CakePHP migration creates the `email_jobs` table. The schema is:

Column	Type	Purpose
id	UUID / PK	Unique job identifier
mailer	VARCHAR(100)	Mailer class name (e.g. User)
action	VARCHAR(100)	Mailer action (e.g. resetPassword)
recipient	VARCHAR(255)	Recipient email address
payload	JSON	Serialised data passed to the mailer action
status	ENUM	pending processing sent retrying failed
attempt_count	TINYINT	How many delivery attempts have been made
last_error	TEXT NULL	Full error message from the last failed attempt
next_retry_at	DATETIME NULL	Earliest time the worker should retry this job
sent_at	DATETIME NULL	Timestamp of confirmed delivery
failed_at	DATETIME NULL	Timestamp when job exceeded max attempts
created_at	DATETIME	When the job was enqueued
updated_at	DATETIME	Last status change timestamp

Database index: (status, next_retry_at) — the worker queries this pair on every loop. Without an index, queue polling degrades linearly as the table grows.

5.2 Email Queue Service

File to create: app/cake/src/Service/EmailQueueService.php

This service is the only new interface that controllers and model callbacks interact with. It replaces the direct `->send()` calls in two files:

```
// ForgotController.php — before
```

```
$this->getMailer('User')->send('resetPassword', [$user, $url]);
```

```
// ForgotController.php — after
```

```
$this->EmailQueueService->enqueueMailerAction('User', 'resetPassword', [  
    'user_id' => $user->id,  
    'url'      => $url,  
    'email'    => $user->email,  
]);
```

```
// UsersTable.php afterSaveCommit — before
```

```
$this->getMailer('User')->send('welcome', [$user]);
```

```
$this->getMailer('User')->send('adminNewUser', [$user]);
```

```
// UsersTable.php afterSaveCommit — after
```

```
$this->EmailQueueService->enqueueMailerAction('User', 'welcome', ['user_id' =>  
$user->id]);  
$this->EmailQueueService->enqueueMailerAction('User', 'adminNewUser', ['user_id' =>  
$user->id]);
```

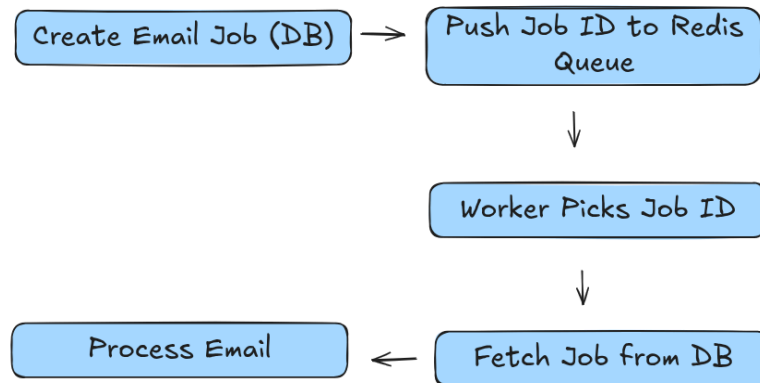
The service will:

- Create a row in **email_jobs** with **status = pending**
- Push the job ID into the Redis queue
- Return immediately, ensuring the request does not wait for SMTP processing

The service will reuse the existing Redis queue pattern (or a shared queue layer), avoiding direct dependency on **TaskComponent** and keeping the design aligned with the current architecture.

If Redis push fails, the job remains **pending** in the database for later processing; however, database persistence failure is treated as a critical error and must be surfaced clearly.

I will use Redis as the fast queue transport, but not as the only storage. The database persists the full job record while Redis carries only the job ID for fast background dispatch.



5.3 Background worker

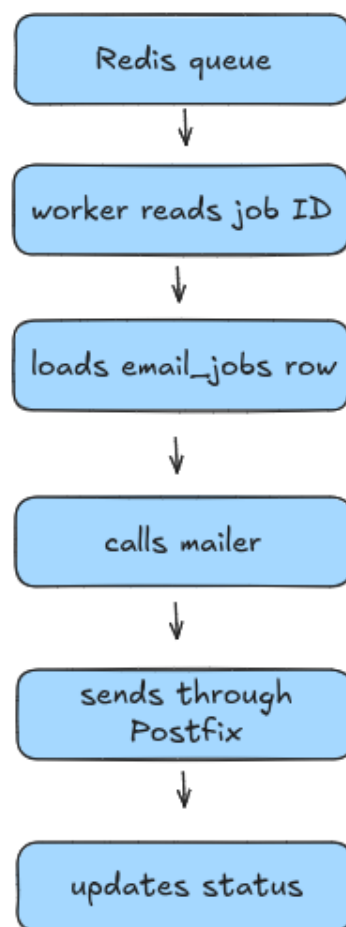
A dedicated **email_worker** container is added to **dev/docker-compose.yml** using the same CakePHP app image, running alongside the existing **app_cake**, **nginx**, **mariadb**, **redis**, and **postfix** services — no new Docker image needed. The container runs with **restart: always** so it recovers automatically from crashes.

File to create: **app/cake/src/Command/EmailWorkerCommand.php**

A CakePHP CLI command implementing the main delivery loop. It runs indefinitely and is managed by Docker. The loop sequence on each iteration:

1. BLPOP from Redis list **email:queue** with a 5-second timeout — blocks efficiently with no busy-wait
2. Load the **email_jobs** row by ID. If not found or not pending, skip
3. Acquire **SELECT FOR UPDATE** on the row to prevent two workers from processing the same job simultaneously
4. Set **status = processing**, increment **attempt_count**, save
5. Reconstruct the mailer payload from the JSON column and load associated ORM records as needed

6. Call the mailer: `$this->getMailer($job->mailer)->send($job->action, $args)`
7. On success: set **status = sent**, **sent_at = now()**, save
8. On transient failure: compute **next_retry_at** using the backoff schedule, set **status = retrying**, save, push ID back to Redis. The job is not immediately pushed back into the same Redis queue.
9. On permanent failure or max attempts exceeded: set **status = failed**, **failed_at = now()**, save — do not requeue
10. Write a heartbeat timestamp to Redis key **email:worker:heartbeat**



Watchdog: On startup and every 5 minutes, the worker scans for rows stuck in **status = processing** with **updated_at** older than the configurable **stuck_job_timeout** (default: 10 minutes). These are reset to **status = pending** and re-pushed to the Redis queue. This handles worker crashes mid-job without requiring manual intervention.

5.4 Docker Configuration

File to modify: dev/docker-compose.yml, dev/docker-compose.mpi.yml
dev/docker-compose.gea.yml

Only one new service will be added to the Docker setup: **email_worker**. It will run the CakePHP CLI worker responsible for processing queued email jobs. Redis, MariaDB, Postfix, and the main application container already exist and will be reused.

```
email_worker:
  build:
    context: ../app/cake
    dockerfile: Dockerfile
    command: bin/cake email_worker
  restart: always
  depends_on:
    - mariadb
    - redis
    - postfix
  environment:
    EMAIL_TRANSPORT_DEFAULT_URL:
"${CDLI_POSTFIX_SMTP_SERVER:+smtp://postfix:25}"
    CF_TURNSTILE_SITEKEY: "${CF_TURNSTILE_SITEKEY}"
    CF_TURNSTILE_SECRET: "${CF_TURNSTILE_SECRET}"
```

No new Docker image is required. The worker uses the same CakePHP application container, so it shares all existing dependencies, configuration, and the ORM layer.

Job scheduling is managed within the **email_worker** using the **next_retry_at** field in the **email_jobs** table. Redis is used for immediate job dispatch, while delayed retries are handled through periodic database scans, removing the need for any external scheduler.

5.5 Retry Logic

The backoff schedule for version 1:

Attempt	Delay before retry	Scenario handled
1	60 seconds	Momentary SMTP hiccup or brief network drop
2	5 minutes	Provider temporarily overloaded or rate-limited
3	15 minutes	Longer upstream outage — last chance before failure
4+	No retry — marked failed	Persistent failure retained for admin review

Transient vs permanent error classification: HTTP 5xx responses, SMTP 4xx, connection timeouts, and DNS failures are transient. relay policy rejections are permanent. The worker catches and classifies exceptions from the mailer layer before deciding whether to requeue.

5.6 Email Template System

- Templates such as **app/cake/templates/email/html/resetpassword.php** and **app/cake/templates/email/html/welcome.php** are simple standalone content blocks. They are not layout-driven and do not share branding, headers, or footers.
- I will create a shared email design system with a new layout file at **app/cake/templates/layout/email/html/default.php** and optional partials such as **app/cake/templates/element/email/header.php**. Each mailer action will pass a title, body content, and optional CTA link into this shared layout.
- Email HTML constraints will be respected throughout — table-friendly structure, inline-compatible styling, mobile-safe widths — since email clients do not behave like browsers. Plain-text templates under **app/cake/templates/email/text/** will be preserved for accessibility and deliverability.

5.7 Admin Dashboard

The index page is filterable by status (pending, sent, failed, retrying), date range, and recipient. Summary widgets at the top show pending count, sent today, failed today, oldest pending job age, and a list of the most recent error messages.

The retry action resets a failed job's status to pending, clears `last_error`, and pushes the job ID to the Redis queue. It uses the original payload — no re-entry of data is required. The action is restricted to admin-role users via the existing CakePHP authorization layer.

Email Monitoring

Track queued, sent, failed, and retried transactional emails from the admin dashboard.

[Back to Admin Dashboard](#)

Pending Current queue state	18	Sent Current queue state	1420	Failed Current queue state	7	Retrying Current queue state	3
--------------------------------	----	-----------------------------	------	-------------------------------	---	---------------------------------	---

Status: All
 Email Type: All
 Date Range: Last 7 days
 Recipient search: Search by recipient
 Filter

Queue health

Operational snapshot for backlog, delivery, and recent failures.

Sent today	Failed today
128	3
Queue backlog	Oldest pending
21	11 min

Recent errors

- SMTP timeout
- Temporary upstream 421 response

Recent email jobs

Transactional emails only: welcome, password reset, and admin notifications.

[Retry failed](#)

Recipient	Type	Status	Created	Retries	Last Error
sonika@example.com	Welcome email	Sent	2026-03-16 10:15	0	None
researcher@example.com	Password reset	Pending	2026-03-16 10:21	0	Queued for worker pickup
admin@cdli.network	Admin new user	Retrying	2026-03-16 10:02	2	SMTP timeout while relaying
curator@example.com	Crowdsourcing request	Failed	2026-03-16 09:48	3	Upstream provider rejected connection

5.8 Rate Limiting

File to create: `app/cake/src/Service/RateLimiterService.php`

The current system applies a basic 5-minute cooldown per user in `ForgotController.php`, which is limited and not configurable.

This will be extended into a **Redis-backed rate limiting system** using the existing **TaskComponent**, without adding new infrastructure. A new **RateLimiterService** will enforce:

- Per-email request limits
- Per-IP request limits
- Configurable time windows (e.g., requests per hour)

The existing cooldown will be retained and complemented by these additional layers. All error messages will remain uniform to prevent account enumeration.

5.10 Health Check System

File to create: `app/cake/src/Command/EmailHealthCommand.php`

The health check covers:

Check	What it measures and when it fails
Redis connectivity	PING/PONG handshake. Fails if Redis is unreachable or if the email:queue key is inaccessible.
Worker heartbeat	Reads email:worker:heartbeat from Redis. Fails if timestamp is older than 2× the loop interval.
Queue backlog	COUNT of email_jobs WHERE status IN (pending, retrying). Warns if > configurable threshold.
Postfix reachability	TCP connect to the Postfix host:port. Fails if connection is refused or times out.
Recent failure rate	COUNT failed jobs in the last hour. Warns if above threshold.
Mail config validity	Checks that CDLI_POSTFIX_SMTP_SERVER and credentials env vars are set and non-empty.

5.11 Testing Strategy

Each component will have unit tests and integration tests. The test approach follows the patterns already established in the CakePHP test suite under tests/.

- EmailQueueService: assert that enqueue writes a DB row with correct fields and pushes to Redis. Assert that Redis failure does not throw and logs correctly.
- Worker loop: use a test mailer action that records invocations. Assert that status transitions are correct for success, transient failure, and permanent failure paths.
- Retry logic: assert that attempt_count increments and next_retry_at advances by the correct backoff interval.
- Watchdog: assert that jobs in processing with old updated_at are reset to pending.
- Rate limiter: assert per-email and per-IP counters reset after the time window. Assert that uniform error messages do not leak address existence.
- Admin retry: assert that a failed job transitions back to pending and appears in the Redis queue.

End-to-end integration tests will use the existing Docker Compose stack to verify the full path from enqueue → worker → Postfix relay for all four email flows (welcome, resetPassword, adminNewUser, adminCrowdsourcingPrivilege).

5.12 Documentation

Documentation will live in the repository alongside the code so it stays in sync with the implementation. Files to update or create:

File to update: **FRAMEWORK_INSTALL.md**

This document will be expanded to include:

- Setup instructions for the `email_worker` service, including how to start, stop, and verify the worker in different environments.
- Required environment variables, along with a clear distinction between local and production configurations.
- Debugging guidance covering how to inspect the `email_jobs` table, read worker logs, and trace an email flow from enqueue to final delivery.
- Instructions for manually retrying failed jobs through the admin dashboard.
- Documentation of common failure scenarios, including: Redis being unreachable, Jobs stuck in processing, Postfix relay rejection, Upstream SMTP rate limiting

Milestones

Milestone 1: Foundation and Architecture Setup (Weeks 1–2)

Deliverables:

- Create the **email_jobs table** using CakePHP migration
- Define schema fields: `status`, `next_retry_at`, `sent_at`, `failed_at`, `attempt_count`
- Add database index on (**status**, **next_retry_at**)
- Implement **EmailQueueService** with **enqueueMailerAction()**
- Integrate **Redis dispatch** using existing queue pattern or shared layer
- Add unit tests for **database writes** and **Redis enqueue flow**

Outcome:

- A complete **job persistence layer** with **queue integration** for async email processing

Milestone 2: Refactoring and Background Processing (Weeks 3–4)

Deliverables:

- **Refactor email flow:** Replace synchronous `->send()` in `ForgotController.php` and `UsersTable.php`, and migrate all email flows (welcome, reset password, admin flows)
- **Worker setup:** Implement `EmailWorkerCommand` and add `email_worker` service in Docker
- **Reliability handling:** Add job locking and watchdog recovery for stuck jobs

Outcome:

- Email delivery **decoupled from request lifecycle** and handled asynchronously

Milestone 3: End-to-End Flow and Retry System (Weeks 5–6)

Deliverables:

- **End-to-end validation:** Test full pipeline (queue → worker → mailer → Postfix) and verify job lifecycle transitions
- **Retry mechanism:** Implement backoff retries with `attempt_count`, `next_retry_at`, and worker-driven requeueing
- **Failure handling:** Classify transient vs permanent errors and validate retry edge cases

Outcome:

- Reliable email delivery with **structured retry** and **failure handling**

Milestone 4: Admin Controls and Template System (Weeks 7–8)

Deliverables:

- **Admin controls:** Build email job management (list, view, retry) with access restrictions and manual retry support
- **Template system:** Create shared email layout and refactor all templates to use it
- **Testing:** Verify rendering across major email clients

Outcome:

- Centralized templates and **admin control** over email jobs

Milestone 5: Monitoring, Rate Limiting, and Policies (Weeks 9–10)

Deliverables:

- **Build monitoring and admin dashboard:** Implement status filters, summary metrics, recent error visibility, and integrate health checks into the email monitoring flow
- **Implement rate limiting and policies:** Develop Redis-based RateLimiterService with per-email and per-IP limits, and enforce sender policy validation in the worker
- **Add testing coverage:** Write tests for rate limiting and related enforcement logic

Outcome:

- Improved **observability**, **system protection**, and **operational control**

Milestone 6: Hardening, Documentation, and Final Delivery (Weeks 11–12)

Deliverables:

- Harden and validate email infrastructure: Configure Postfix securely, enforce TLS-based SMTP communication, and add connectivity checks
- Implement monitoring and documentation: Build EmailHealthCommand CLI and complete setup, debugging, failure, and health check documentation
- Final testing and stabilization: Perform end-to-end testing, followed by code cleanup, review, and final polish

Outcome:

- Production-ready, **secure**, and **well-documented** email system

Timeline

Community Bonding Period

Explore the full CDLI codebase focusing on the email and task infrastructure. Understand the existing worker and task patterns. Finalise the email_jobs schema and backoff configuration with mentors. Set up local development environment with all Docker services running and a test SMTP account.

Week	Deliverable	Detail
Week 1	Email job model & migration	Create email_jobs table via CakePHP migration. Define all columns including next_retry_at, sent_at, failed_at, and status ENUM. Add DB index on (status, next_retry_at).
Week 2	EmailQueueService & Redis integration	Implement EmailQueueService with enqueueMailerAction(). Wire to existing TaskComponent for Redis dispatch. Write unit tests for DB write and Redis push paths.
Week 3	Refactor current email flows	Replace ->send() calls in ForgotController.php and UsersTable.php. Migrate all four flows: welcome, resetPassword, adminNewUser, adminCrowdsourcingPrivilege. Verify no regression.
Week 4	Background worker & Docker	Build EmailWorkerCommand as CakePHP CLI. Add email_worker service to docker-compose.yml. Implement SELECT FOR UPDATE locking and watchdog for stuck jobs.
Week 5	End-to-end integration testing	Test full path: enqueue → worker → UserMailer → Postfix for all four flows. Fix integration issues. Verify status transitions and DB state.
Week 6	Retry logic & failure handling	Implement exponential backoff with attempt_count and next_retry_at. Classify transient vs permanent errors. Test all retry state transitions.
Week 7	Manual retry & admin controls	Build Admin/EmailJobsController with index, view, and retry actions. Restrict retry to admin

Week	Deliverable	Detail
		role. Test retry resets status and requeues to Redis.
Week 8	Email template system	Create shared layout at templates/layout/email/html/default.php. Refactor existing templates to use shared layout. Verify rendering in major email clients.
Week 9	Admin dashboard & analytics	Complete Admin/EmailJobsController UI with status filters and summary widgets. Build Admin/EmailHealthController with all six health checks.
Week 10	Rate limiting & sender policy	Implement Redis-backed RateLimiterService with per-email and per-IP limits. Enforce sender policy validation in the worker. Write tests for rate limit counter resets.
Week 11	Postfix hardening & health CLI	Enforce TLS to upstream SMTP. Add Postfix reachability check. Implement EmailHealthCommand CLI. Begin documentation in FRAMEWORK_INSTALL.md.
Week 12	Testing, documentation & polish	Complete setup guide, debugging guide, and failure scenario docs. Final end-to-end test run. Fix any remaining issues. Code review and cleanup.

Contributions

- **MR - 1148 :**
Refactored email templates for clarity and consistency, and fixed a typo in the registration subject.
Improved HTML email layout rendering to better support multiline content.
- **MR - 1151 (merged) :**
Fixed broken publication upload links and improved admin UI labeling for clarity. Maintained the existing upload workflow without altering functionality.

Experience

I have experience working across backend and full-stack development, with a strong focus on building scalable and user-centric applications. I have worked with technologies including **Python, HTML, CSS, Javascript, Django, PHP, Flutter, MERN and Git**, which has helped me understand both development and deployment workflows.

As a **Backend Intern**, I worked extensively with **Django**, where I gained hands-on experience in building APIs, handling backend logic, and working with databases, helped me strengthen my understanding of backend architecture, request handling, and real-world application development.

The list of projects I have worked on:

Sarkar Suvidha : A multilingual AI platform integrating WhatsApp, voice, and language-detecting AI bots to help citizens discover and access personalized government schemes seamlessly.

Chat App: A real-time chat application built using webhooks to enable instant messaging between users.

DoLearn: An AI-powered learning agent that generates personalized roadmaps and tracks user progress using online resources.

Trip Tracker: A collaborative app for tracking shared expenses and preserving travel memories as a digital diary.

Why me?

I believe I am a strong fit for this project because it sits at the intersection of backend engineering, data modeling, queue-based systems, and reliability-focused development — which are exactly the areas I want to grow in. My prior experience with Python, Django, REST APIs, relational databases, and background job systems gives me a solid foundation for building an asynchronous email pipeline in CakePHP. I also have direct hands-on experience with PHP and the CDLI codebase specifically through my merged contributions.

Beyond that, I did not approach this proposal as an idea. I set up the CDLI framework locally, read through `ForgotController.php`, `UsersTable.php`, `TaskComponent.php`, and the existing mailer and template files, and shaped every section of this proposal around what I actually found in the code — not around what I assumed was there. I engaged with the community, submitted merge requests, incorporated mentor feedback, and kept refining the design until it fit naturally into the existing architecture. Because of that preparation, I feel ready not just to understand the problem, but to implement a solution that a future maintainer would be comfortable extending.

Use Of LLM

- I use LLM tool Claude as assistive tools for brainstorming, improving documentation, checking wording, and reviewing design alternatives.
- I may also use them for small code-level suggestions or test ideas, but not as a substitute for understanding the codebase. For this proposal, AI helped me structure sections, refine explanations, and check consistency.
- while the actual technical content was based on reading the CDLI repository, reviewing the current email and task infrastructure, and incorporating mentor feedback.
- During the project, I would continue to use AI in a limited and careful way for brainstorming, summarization, and review support, while verifying all technical decisions through direct code reading, testing, and mentor discussion.

Other Commitments During Summer

I do not have any major commitments during the GSoC period.

I will dedicate a minimum of **35–40 hours per week** to the project and remain actively engaged with mentors and the community.